

Program 9 Working with Maven to Gradle Migration

Overview

This lab demonstrates how to migrate a Java project from Maven to Gradle. The project, named MyMavenToGradle, was first created using Maven's archetype plugin, compiled and tested, and then converted to a Gradle build system using the `gradle init --type pom` command. The final project was pushed to a GitHub repository.

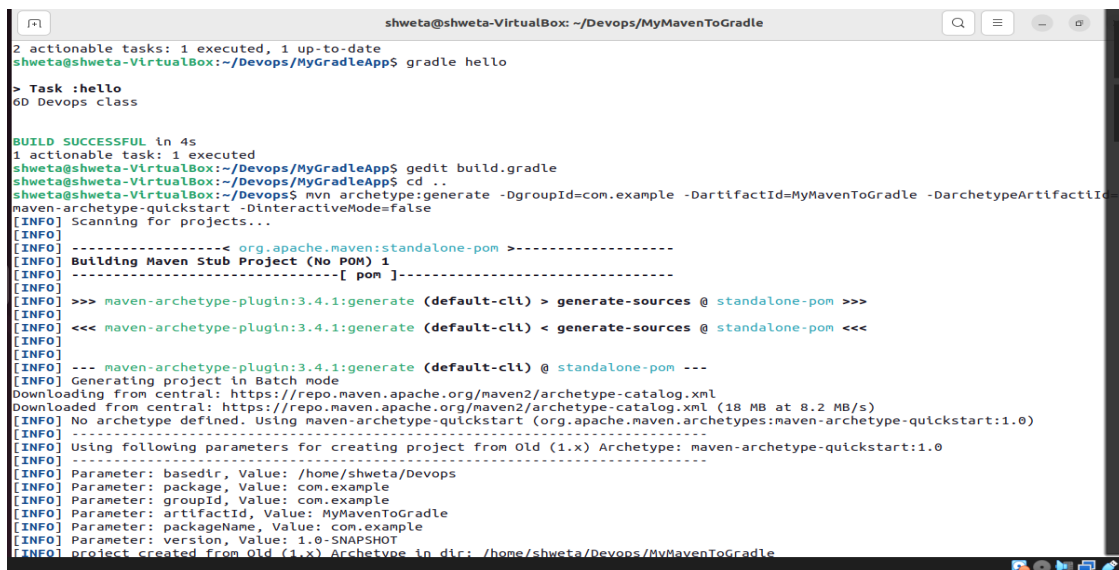
Step-by-Step Process

Step 1: Create Maven Project

A Maven project was generated from the command line using the Maven archetype quickstart plugin with the following parameters:

```
mvn archetype:generate -DgroupId=com.example \
-DartifactId=MyMavenToGradle \
-DarchetypeArtifactId=maven-archetype-quickstart \
-DinteractiveMode=false
```

The project was created in the `~/Devops` directory with the following structure:



```
shweta@shweta-VirtualBox: ~/Devops/MyMavenToGradle
2 actionable tasks: 1 executed, 1 up-to-date
shweta@shweta-VirtualBox:~/Devops/MyGradleApp$ gradle hello

> Task :hello
60 Devops class

BUILD SUCCESSFUL in 4s
1 actionable task: 1 executed
shweta@shweta-VirtualBox:~/Devops/MyGradleApp$ gedit build.gradle
shweta@shweta-VirtualBox:~/Devops/MyGradleApp$ cd ..
shweta@shweta-VirtualBox:~/Devops$ mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenToGradle -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
[INFO] Scanning for projects...
[INFO]
[INFO] -----< org.apache.maven:standalone-pom >-----
[INFO] Building Maven Stub Project (No POM) 1
[INFO] -----[ pom ]-----
[INFO]
[INFO] >>> maven-archetype-plugin:3.4.1:generate (default-cli) > generate-sources @ standalone-pom >>>
[INFO]
[INFO] <<< maven-archetype-plugin:3.4.1:generate (default-cli) < generate-sources @ standalone-pom <<<
[INFO]
[INFO] --- maven-archetype-plugin:3.4.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
[INFO] Downloading from central: https://repo.maven.apache.org/maven2/archetype-catalog.xml
[INFO] Downloaded from central: https://repo.maven.apache.org/maven2/archetype-catalog.xml (18 MB at 8.2 MB/s)
[INFO] No archetype defined. Using maven-archetype-quickstart (org.apache.maven.archetypes:maven-archetype-quickstart:1.0)
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-quickstart:1.0
[INFO]
[INFO] Parameter: basedir, Value: /home/shweta/Devops
[INFO] Parameter: package, Value: com.example
[INFO] Parameter: groupId, Value: com.example
[INFO] Parameter: artifactId, Value: MyMavenToGradle
[INFO] Parameter: packageName, Value: com.example
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: /home/shweta/Devops/MyMavenToGradle
```

Step 2: Edit pom.xml and Examine Maven Structure

After creation, the pom.xml was opened using gedit to review the Maven build configuration. The project tree was displayed to confirm the directory layout:

- pom.xml – Maven build descriptor
- src/main/java/com/example/App.java – Main application class
- src/test/java/com/example/AppTest.java – JUnit test class

Step 3: Build the Maven Project (mvn install)

The project was compiled and packaged using Maven's install lifecycle command:

```
mvn install
```

This ran the full Maven lifecycle: compile → test → package → install. Maven's Surefire plugin ran the tests and the maven-jar-plugin packaged the application. The JAR was installed to the local .m2 repository.

```
shweta@shweta-VirtualBox: ~/Devops/MyMavenToGradle
[INFO] skip non existing resourceDirectory /home/shweta/Devops/MyMavenToGradle/src/test/resources
[INFO] --- naven-compiler-plugin:3.8.1:testCompile (default-testCompile) @ MyMavenToGradle ---
[INFO] Changes detected - recompiling the module!
[WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
[INFO] Compiling 1 source file to /home/shweta/Devops/MyMavenToGradle/target/test-classes
[INFO] --- naven-surefire-plugin:2.22.2:test (default-test) @ MyMavenToGradle ---
[INFO]
[INFO] T E S T S
[INFO] Running com.example.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.029 s - in com.example.AppTest
[INFO] Results:
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] --- naven-jar-plugin:3.1.0:jar (default-jar) @ MyMavenToGradle ---
[INFO] Building jar: /home/shweta/Devops/MyMavenToGradle/target/MyMavenToGradle-1.0-SNAPSHOT.jar
[INFO] --- naven-install-plugin:2.4:install (default-install) @ MyMavenToGradle ---
[INFO] Installing /home/shweta/Devops/MyMavenToGradle/target/MyMavenToGradle-1.0-SNAPSHOT.jar to /home/shweta/.m2/repository/com/example/MyMavenToGradle/1.0-SNAPSHOT/MyMavenToGradle-1.0-SNAPSHOT.jar
[INFO] Installing /home/shweta/Devops/MyMavenToGradle/pom.xml to /home/shweta/.m2/repository/com/example/MyMavenToGradle/1.0-SNAPSHOT/MyMavenToGradle-1.0-SNAPSHOT.pom
[INFO] BUILD SUCCESS
[INFO] Total time: 30.950 s
[INFO] Finished at: 2026-03-25T09:09:05+05:30
[INFO]
shweta@shweta-VirtualBox:~/Devops/MyMavenToGradle$ ls
pom.xml src target
shweta@shweta-VirtualBox:~/Devops/MyMavenToGradle$ gradle init --type pom.xml
<-----> 0%
```

Step 4: Convert Maven to Gradle using gradle init

With the Maven build successful, the Gradle initialization command was run inside the MyMavenToGradle directory to convert the project:

```
gradle init --type pom
```

Note: The first attempt used --type pom.xml which failed with an error because that is not a valid type. The correct flag is --type pom.

```

2 actionable tasks: 2 executed
shweta@shweta-VirtualBox:~/Devops/MyMavenToGradle$ gradle init --type pom

> Task :init
Maven to Gradle conversion is an incubating feature.

BUILD SUCCESSFUL in 1m 8s
2 actionable tasks: 1 executed, 1 up-to-date
shweta@shweta-VirtualBox:~/Devops/MyMavenToGradle$ ls
build.gradle  gradle  gradlew  gradlew.bat  pom.xml  settings.gradle  src  target
shweta@shweta-VirtualBox:~/Devops/MyMavenToGradle$

```

The gradle init --type pom command read the existing pom.xml and auto-generated the Gradle build files. After conversion, the project contained both Maven and Gradle build files:

- – **Generated Gradle build script (equivalent to pom.xml)** build.gradle
- – **Project name and subproject settings** settings.gradle
- – **Gradle wrapper scripts for Unix and Windows** gradlew / gradlew.bat
- – **Gradle wrapper JAR and properties** gradle/wrapper/

Step 5: Inspect the Converted Project Tree

After conversion, the full project tree was displayed to confirm all files were generated correctly:

```

Failed to register: timeout was reached
shweta@shweta-VirtualBox:~/Devops/MyMavenToGradle$ tree
.
├── build.gradle
├── gradle
│   └── wrapper
│       ├── gradle-wrapper.jar
│       └── gradle-wrapper.properties
├── gradlew
├── gradlew.bat
├── pom.xml
├── settings.gradle
├── src
│   ├── main
│   │   ├── java
│   │   │   ├── com
│   │   │   │   └── example
│   │   │       └── App.java
│   └── test
│       ├── java
│       │   ├── com
│       │   │   └── example
│       │       └── AppTest.java
├── target
│   ├── classes
│   │   ├── com
│   │   │   └── example
│   │       └── App.class
│   ├── generated-sources
│   │   ├── annotations
│   │   ├── generated-test-sources
│   │   └── test-annotations
│   ├── maven-archiver
│   │   └── pom.properties
│   ├── maven-status
│   │   └── maven-compiler-plugin
│   │       ├── compile
│   │       │   ├── default-compile
│   │       │   │   ├── createdFiles.lst
│   │       │   │   └── inputFiles.lst
│   │       └── testCompile
│   │           ├── default-testCompile
│   │           │   ├── createdFiles.lst
│   │           │   └── inputFiles.lst
│   └── MyMavenToGradle-1.0-SNAPSHOT.jar
├── surefire-reports
│   ├── com.example.AppTest.txt
│   └── TEST-com.example.AppTest.xml
├── test-classes
│   ├── com
│   │   └── example
│   └── AppTest.class
└── 38 directories, 19 files
[1]+  Killed                  gedit pom.xml
shweta@shweta-VirtualBox:~/Devops/MyMavenToGradle$ =

```

The project now contained both the original Maven target/ directory (from the previous mvn install) and the newly generated Gradle build files. The source structure remained unchanged.

Step 6: Push to GitHub

The project was then pushed to a remote GitHub repository. The following Git commands were executed:

```
git init
git add .
git commit -m "first commit"
git remote add origin https://github.com/Shweta311204/MyMavenToGradle.git
git remote set-url origin https://ghp_...@github.com/Shweta311204/MyMavenToGradle.git
git push -u origin master
```

Set Up Jenkins Pipeline for MyMavenToGradle

Step 1: To deploy **MyMavenToGradle**, open the terminal, cd into the **MyMavenToGradle**

folder, create a **Jenkinsfile** containing the pipeline code, and then commit and push it to GitHub. pipeline {

```
agent any // Use any available agent tools {
```

```
gradle 'Gradle' // Ensure this matches the name configured in Jenkins jdk 'JDK'
}
```

```
stages {
```

```
stage('Checkout') { steps {
```

```
git branch: 'master', url: 'https://github.com/Shweta311204/MyMavenToGradle.git'
}
}
```

```
stage('Build') { steps {
```

```
sh 'gradle build' // Run Gradle build
}
```

```
}

stage('Test') { steps {

sh 'gradle test' // Run unit tests
}
}

stage('Run Application') { steps {

// Start the JAR application sh 'java -jar build/libs/*.jar'
}
}

post {

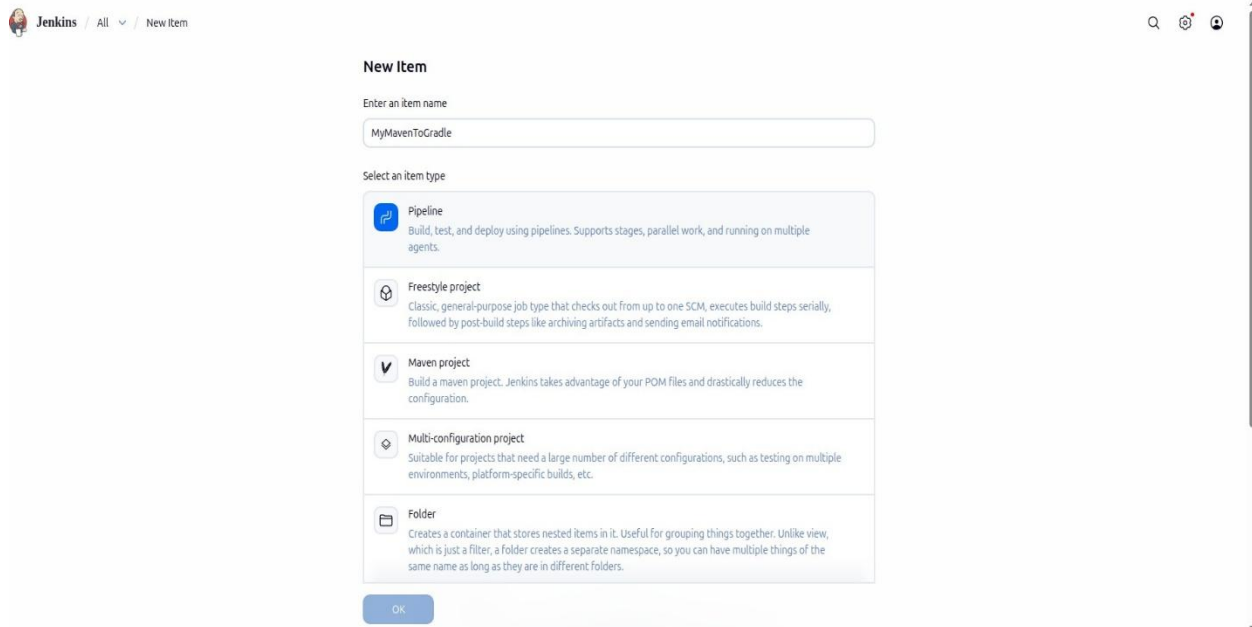
success {

echo 'Build and deployment successful!'
}

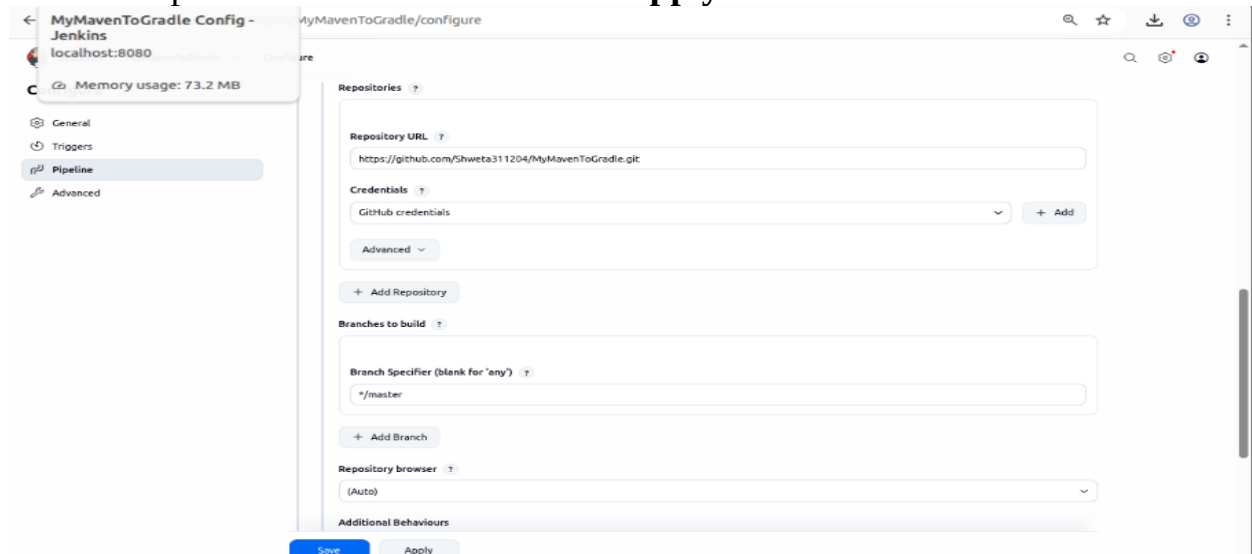
failure {

echo 'Build failed!'
}
}
}
```

Step 2: To deploy **MyMavenToGradle**, you first need to create a Jenkins pipeline job that will use the Jenkinsfile from your project repository. Then, go to the Jenkins home page and click on “**New Item**” to create a job, enter the item name (e.g., *MyMavenToGradle*), and select **Pipeline** as the project type. This option allows you to build, test, and deploy applications using a Jenkinsfile. After selecting the pipeline type, click “**OK**” to proceed to the job configuration page.



Step 3: In **MyMavenToGradle**, go to the **Pipeline** section and under **Definition**, select **Pipeline script from SCM**. Choose **Git** as the SCM, and under **Repository URL**, paste the GitHub URL of **MyMavenToGradle**. In the **Credentials** field, select the previously created credential (e.g., *GitHub Credential*). Finally, in **Script Path**, enter the name of the Jenkinsfile you created in the MyMavenToGradle folder and pushed to GitHub. Then click **Apply** and **Save** to finalize the



configuration.

Step 4: After clicking **Apply** and **Save**, go back to **MyMavenToGradle** in Jenkins and click **Build Now**. If you get an error like:

Error: no main manifest attribute, in build/libs/*.jar or

Error: Could not find or load main class com.example.App

this error occurs when the generated JAR file does not contain the required **Main-Class entry in the MANIFEST.MF file**. Because of this, Java does not know which class should be executed when running the JAR file.

Even though the project builds successfully using Gradle, the application fails at runtime because Gradle by default creates a **non-executable JAR file** without specifying the entry point.

To solve this issue, we explicitly define the Main Class inside the build.gradle file so that the JAR becomes executable and Jenkins can run it properly.

Replace the existing configuration with the following:

```
jar {  
  
    manifest {  
  
        attributes(  
  
            'Main-Class': 'com.example.App'  
        )  
    }  
}
```

Instead of hardcoding the JAR name, we used:

java -jar build/libs/*.jar

This works because Gradle generates the JAR file name dynamically, and *.jar automatically selects the correct generated JAR file from the build/libs directory. This ensures the correct JAR is executed even if the file name changes due to versioning or build configuration.

Step 5: After making the changes, go back to **MyMavenToGradlePipelineApp** in Jenkins and click **Build Now**. The build should complete successfully without any errors.

This works because adding the **Main-Class** in build.gradle makes the JAR executable, so Java knows the entry point (com.example.App). Jenkins can then run the application successfully using `java -jar build/libs/*.jar` without errors like no main manifest attribute.

